Dossier de rendu — GitOps & CI/CD

Auteur : Maxime MAILLARY • **Date :** Novembre 2026 • **Dépôt GitHub :** github.com/maximemlly/tasks-api

---

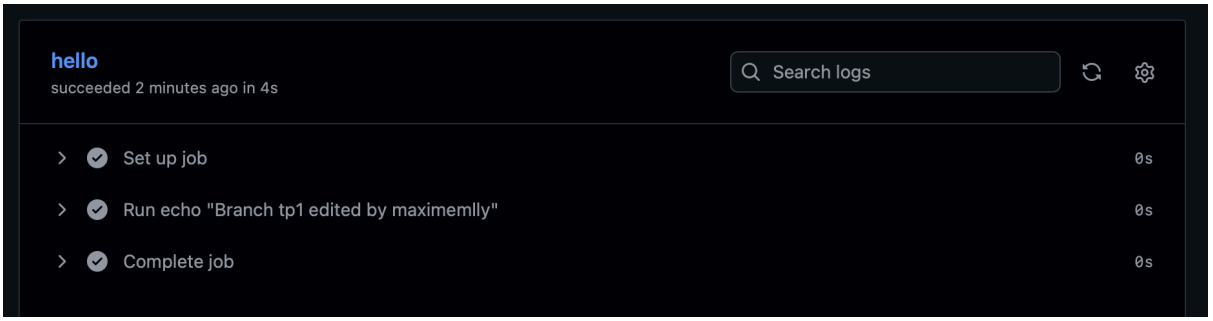
TP 1 : Votre première CI avec GitHub Actions

1. Objectif du TP

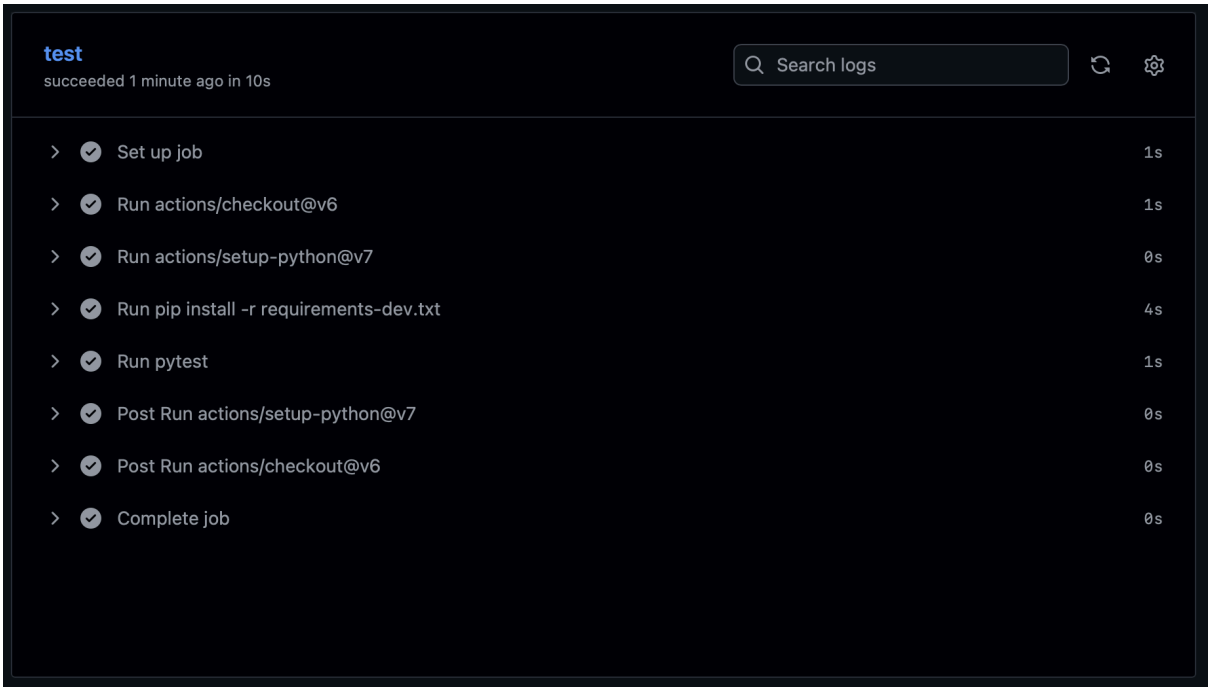
Ce premier TP vise à mettre en place une chaîne d'intégration continue (CI) automatisée avec GitHub Actions sur le projet tasks-api. À chaque push sur le dépôt, un workflow installe l'environnement Python et exécute automatiquement la suite de tests pytest afin de détecter immédiatement toute régression avant la fusion d'une Pull Request sur la branche principale.

2. Captures d'écran

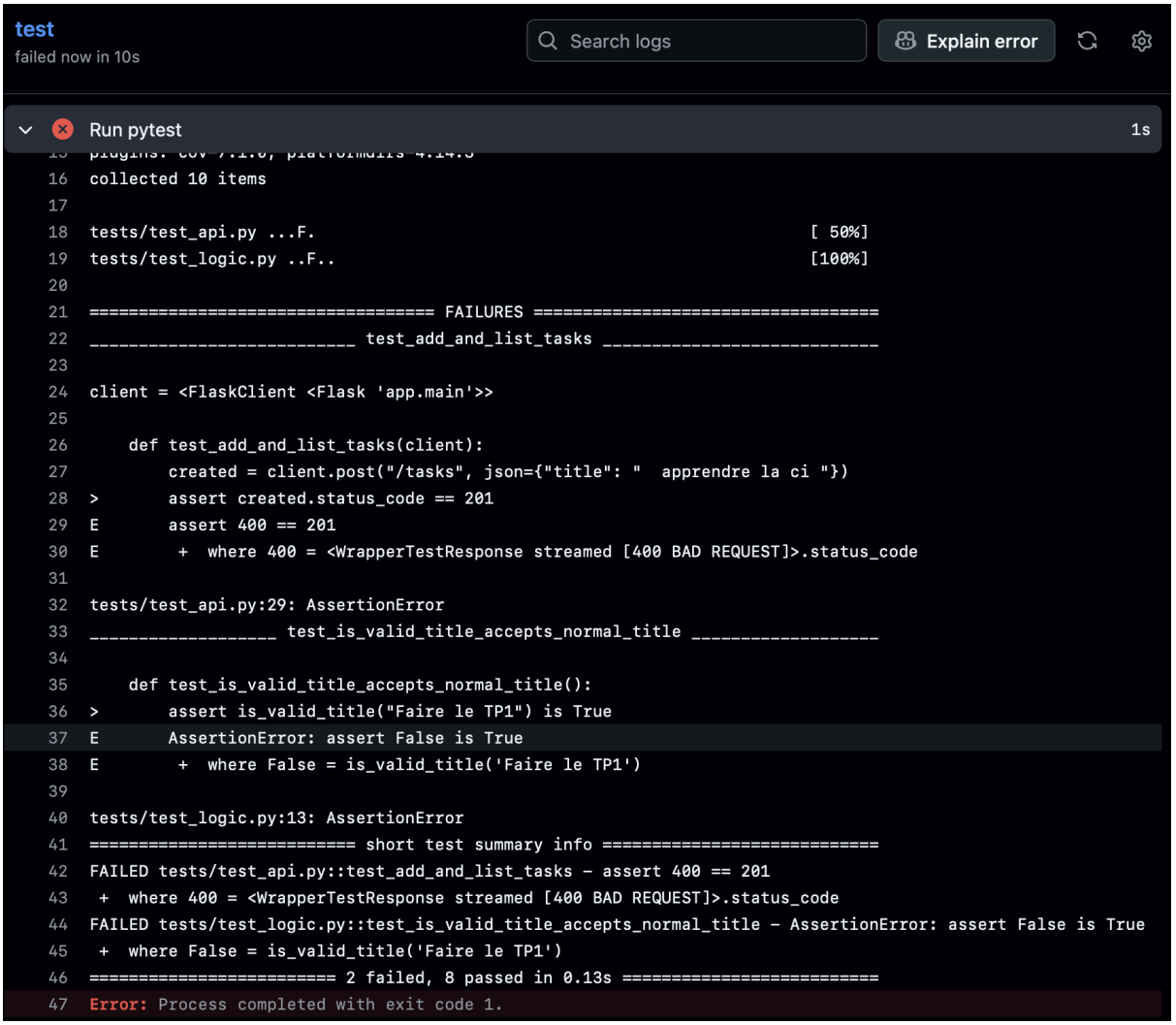
Capture TP1-1 : Exécution réussie du workflow « hello » en 4 secondes, affichant le nom de la branche (tp1) et l'auteur du commit (maximemlly).



Capture TP1-2 : Exécution réussie du workflow « ci.yml » (job test) avec le détail des étapes : récupération du code (checkout@v6), installation de Python 3.12 (setup-python@v7), installation des dépendances et lancement de pytest.



Capture TP1-3 : Run en échec (rouge) suite à la modification volontaire de MAX_TITLE_LENGTH à 5 dans app/logic.py, avec l'étape « Run pytest » dépliée montrant les tests en erreur.



Capture TP1-4 : Retour au vert des workflows « ci » et « hello » sur le commit d600e90 après rétablissement de MAX_TITLE_LENGTH à 80.



Capture TP1-5 : Pull Request ouverte depuis la branche tp1 vers main avec les vérifications (checks) de la CI au vert avant la fusion.

[À insérer : capture d'écran TP1-5]

Capture TP1-6 : Pull Request fusionnée (Merged) dans la branche main une fois les tests validés.

[À insérer : capture d'écran TP1-6]

Capture TP1-7 : Page du dépôt GitHub affichant le fichier README.md avec le badge de statut CI « passing ».

[À insérer : capture d'écran TP1-7]

3. Réponses aux questions

Question 1 : Combien de temps a duré le run « Hello » ? Où voit-on les logs de chaque étape ?

Le run a duré 4 à 5 secondes. On voit les logs détaillés de chaque étape dans l'onglet « Actions » du dépôt GitHub, en cliquant sur le run concerné, puis sur le job « hello » et en dépliant chacune des étapes affichées.

Question 2 : Quel test a échoué dans la partie 3, et à quelle ligne ? Qu'est-ce qui a permis de le trouver rapidement ?

Dans la partie 3, le test unitaire de logique test_is_valid_title_accepts_normal_title (défini à la ligne 12 de tests/test_logic.py, assertion en échec à la ligne 13 : AssertionError: assert False is True) ainsi que le test d'API test_add_and_list_tasks (ligne 29 de tests/test_api.py : assert 400 == 201) ont échoué. On a pu identifier la cause immédiatement grâce aux logs de l'étape « Run pytest » dans GitHub Actions, où pytest affiche la section FAILURES et le résumé indiquant le fichier exact, le numéro de ligne et la valeur retournée.

Question 3 : Pourquoi attend-on que la CI soit verte avant de fusionner une pull request ?

On attend que la CI soit verte avant de fusionner une pull request car elle permet de s'assurer que le code est de bonne qualité, qu'il respecte les conventions établies et que tous les tests passent. Cela évite d'introduire des régressions ou du code cassé sur la branche principale (main).

4. Difficultés rencontrées

Aucune difficulté bloquante majeure n'a été rencontrée durant ce premier TP. La prise en main de la syntaxe YAML des workflows GitHub Actions et la lecture des logs d'exécution ont permis d'identifier et de corriger rapidement l'erreur introduite à la partie 3.

Dossier de rendu — GitOps & CI/CD

Auteur : Maxime MAILLARY • **Date :** Novembre 2026 • **Dépôt GitHub :** github.com/maximemlly/tasks-api

---

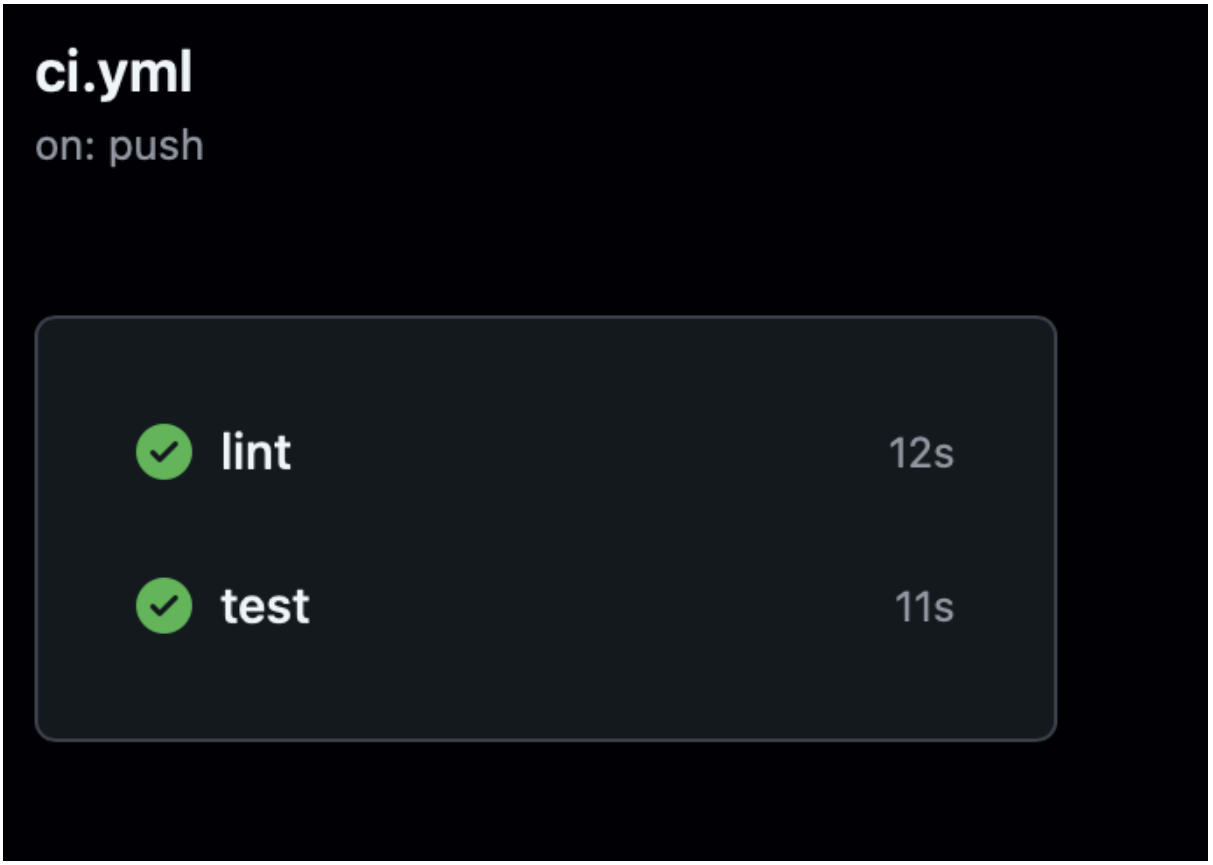
TP 2 : Une CI solide

1. Objectif du TP

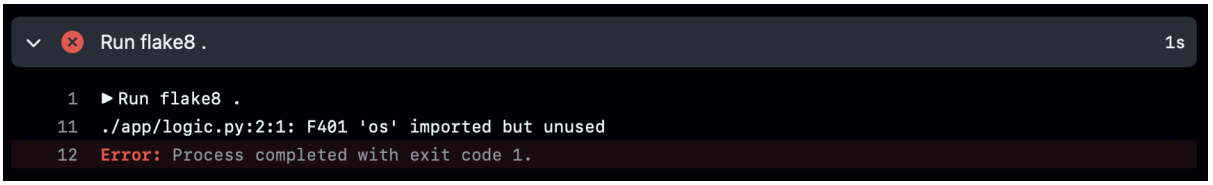
Ce deuxième TP a pour objectif d'enrichir la chaîne d'intégration continue du projet tasks-api pour la rendre complète et robuste : ajout de l'analyse statique et du formatage (flake8 et black), exécution des tests sur plusieurs versions de Python (matrix 3.11 à 3.13) avec seuil de couverture à 80 %, mise en cache des dépendances pip et publication du rapport de couverture en artefact. Il aborde également la protection de la branche main, la résolution de conflits Git, la gestion sécurisée des secrets et le diagnostic de pipelines en échec.

2. Captures d'écran

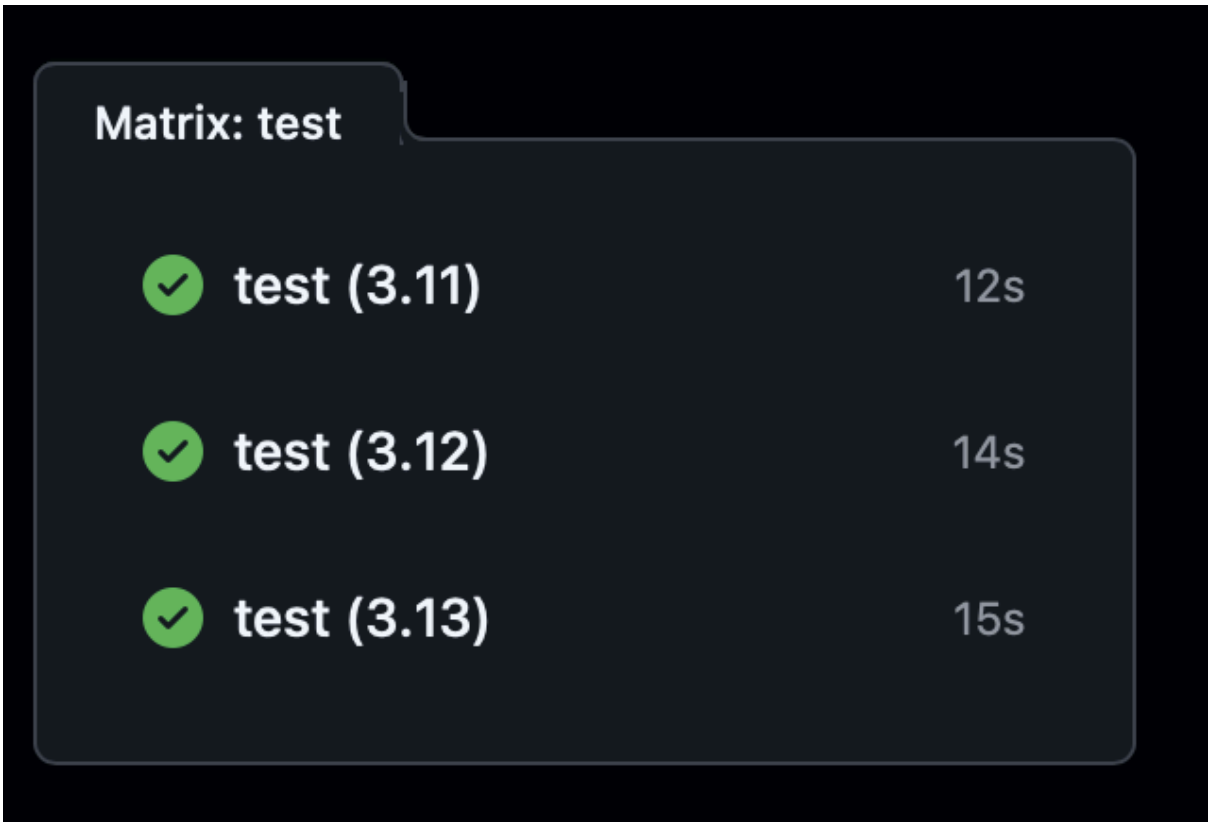
Capture TP2-1 : Exécution réussie du workflow « ci.yml » montrant les deux jobs « lint » (12 s) et « test » (11 s) au vert.



Capture TP2-2 : Job « lint » en échec (rouge) avec l'étape « Run flake8 . » dépliée signalant l'import inutilisé (« ./app/logic.py:2:1: F401 'os' imported but unused »).



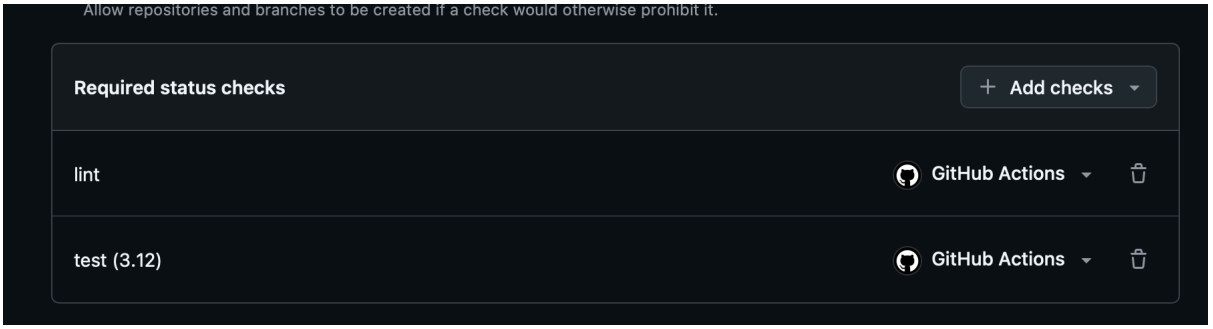
Capture TP2-3 : Liste des 3 jobs de la matrice (« test (3.11) », « test (3.12) » et « test (3.13) ») exécutés avec succès en parallèle.



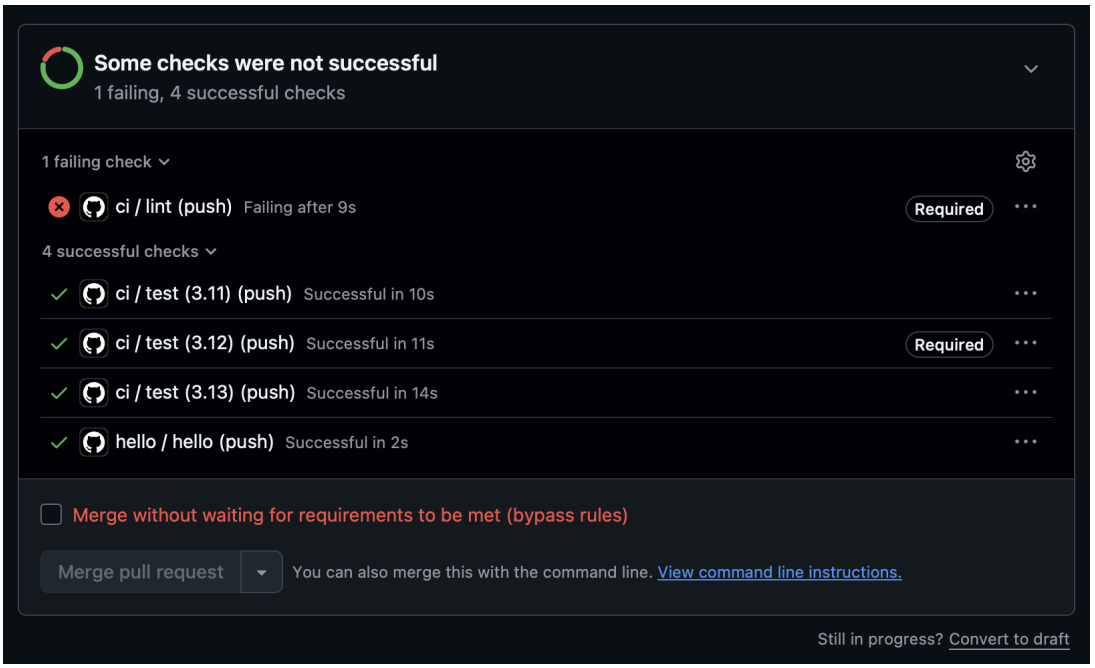
Capture TP2-4 : Étape « Run actions/setup-python@v7 » du second run confirmant la restauration du cache pip (« Cache hit » / « Cache restored successfully »).



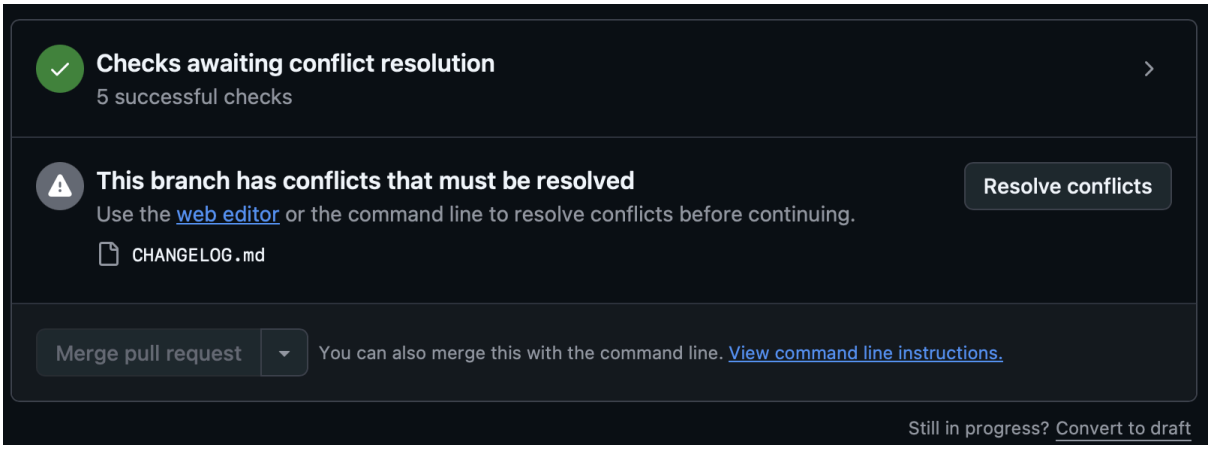
Capture TP2-5 : Page de configuration de la règle de protection de la branche main avec les vérifications obligatoires (« lint » et « test (3.12) »).



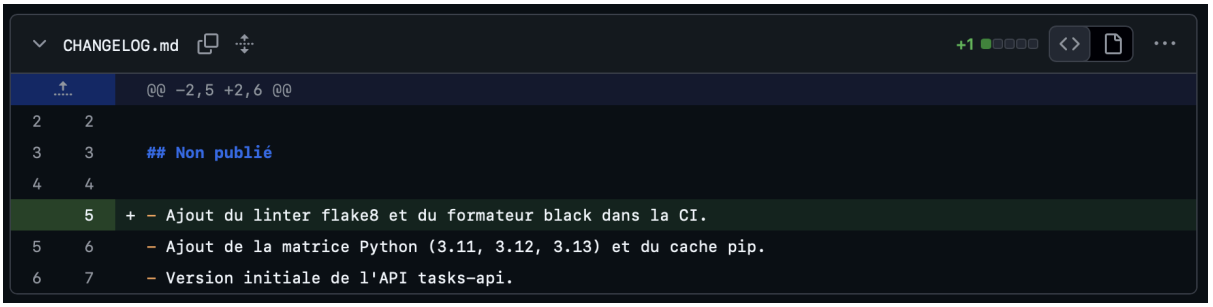
Capture TP2-6 : Pull Request dont la fusion vers main est bloquée par une vérification obligatoire en échec.



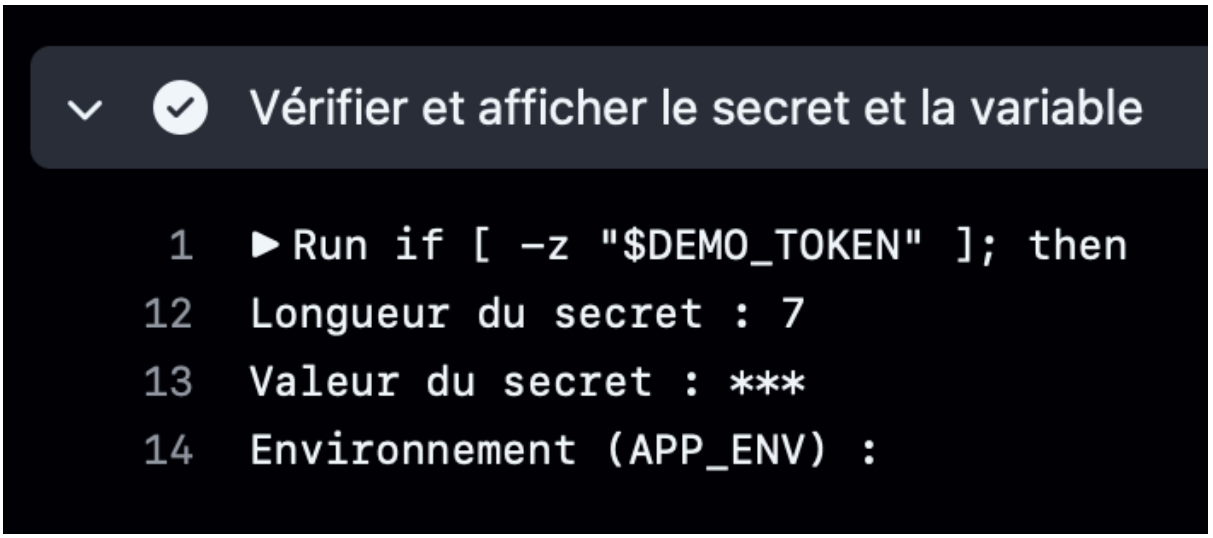
Capture TP2-7 : Pull Request affichant le message de conflit (« This branch has conflicts that must be resolved ») sur CHANGELOG.md.



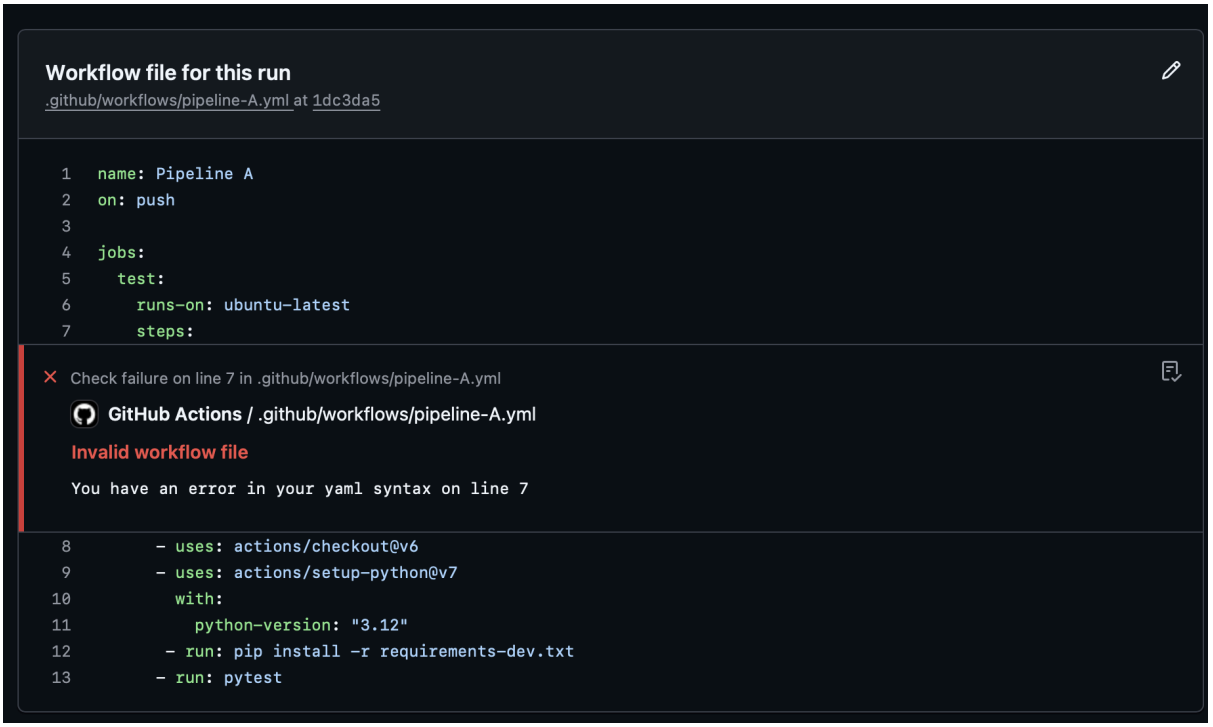
Capture TP2-8 : Fichier CHANGELOG.md final sur la branche main contenant les deux entrées conservées après résolution du conflit.

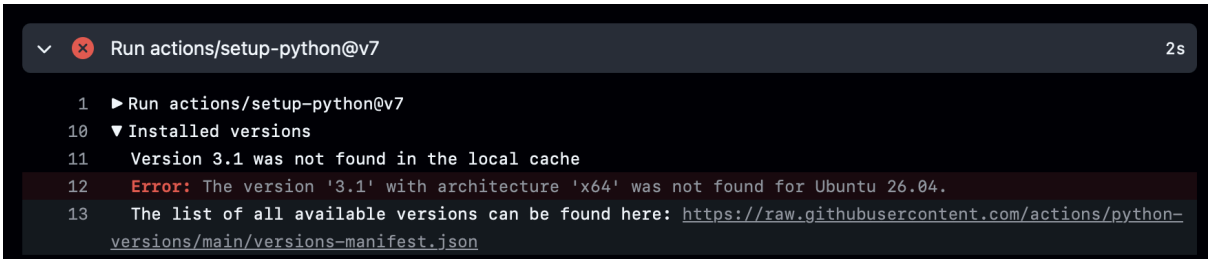


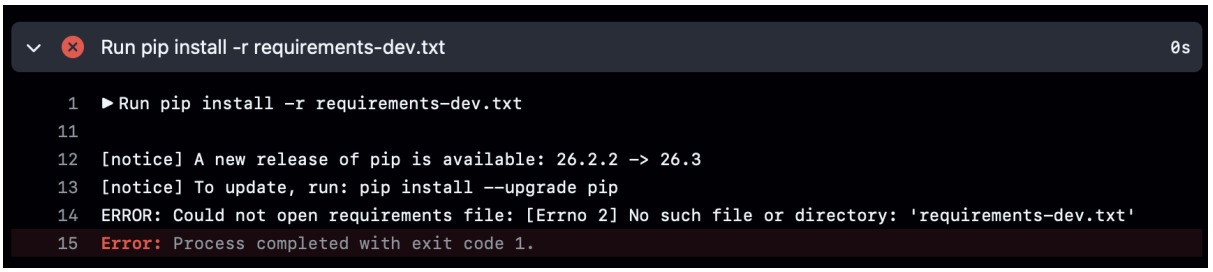
Capture TP2-9 : Logs du workflow secret-demo.yml où la valeur du secret DEMO_TOKEN est masquée par « *** » tandis que sa longueur et la variable APP_ENV sont affichées.



Captures TP2-10A, TP2-10B, TP2-10C : Runs en échec de chacun des trois pipelines à réparer (A, B et C).







Capture TP2-11 : Liste des runs dans GitHub Actions montrant les trois pipelines A, B et C réparés et passés au vert.



3. Réponses aux questions

Question 1 (Partie 1) : Quelle différence entre un linter (flake8) et un formateur (black) ?

Un linter (flake8) analyse le code statiquement pour détecter des erreurs de syntaxe, des imports ou variables inutilisés et le non-respect des conventions PEP 8, tandis qu'un formateur (black) s'occupe uniquement de la mise en forme visuelle du code (indentation, guillemets, retours à la ligne) pour garantir un style uniforme.

Question 2 (Partie 1) : Pourquoi séparer lint et test en deux jobs ?

Séparer lint et test en deux jobs permet de les exécuter en parallèle (gain de temps) et d'isoler les responsabilités : en cas d'échec, on voit immédiatement dans GitHub Actions s'il s'agit d'un problème de style/qualité du code (lint) ou d'une régression fonctionnelle dans les tests unitaires (test).

Question 3 (Partie 2) : Pourquoi tester sur plusieurs versions de Python ?

Tester sur plusieurs versions de Python (3.11, 3.12 et 3.13) grâce à une matrix permet de s'assurer que l'application et ses dépendances fonctionnent correctement sur l'ensemble des environnements supportés et d'anticiper d'éventuelles incompatibilités ou dépréciations.

Question 4 (Partie 2) : Que gagne-t-on avec le cache ?

Le cache pip évite de retélécharger toutes les bibliothèques Python à chaque exécution du workflow lorsque requirements-dev.txt n'a pas changé, ce qui réduit la durée d'installation des dépendances et accélère les runs de CI.

Question 5 (Partie 2) : Un artefact, c'est quoi ?

Un artefact est un fichier (ou ensemble de fichiers) produit lors de l'exécution d'un job (comme le rapport coverage.xml, un binaire compilé ou des logs) que GitHub Actions sauvegarde après la destruction du runner éphémère afin de pouvoir le télécharger ou le partager avec les jobs suivants.

Question 6 (Partie 3) : Pourquoi interdire de pousser directement sur main ?

Interdire les push directs sur main oblige chaque modification à passer par une Pull Request et par les vérifications automatiques obligatoires (lint et tests). Cela garantit que personne ne peut contourner la CI et que la branche principale reste toujours stable et déployable.

Question 7 (Partie 4) : Pourquoi Git ne peut-il pas résoudre ce conflit seul ? Qu'est-ce qui a vérifié le code après la résolution ?

Git ne peut pas résoudre ce conflit automatiquement car les deux branches ont modifié exactement la même ligne de CHANGELOG.md à partir du même commit de base : seul un humain peut décider s'il faut conserver l'une des deux lignes ou les deux, et dans quel ordre. Après la résolution manuelle et le push, c'est la CI (GitHub Actions) qui a revérifié automatiquement le code et les tests avant d'autoriser la fusion.

Question 8 (Partie 5) : Pourquoi GitHub masque-t-il le secret ? Que se passe-t-il si on le commite dans le dépôt ? Secret ou variable : comment choisir ?

GitHub masque automatiquement la valeur des secrets par *** dans les logs pour éviter toute fuite accidentelle d'identifiants sensibles, surtout sur un dépôt public où tout le monde peut lire les journaux d'exécution. Si l'on commite un secret en clair dans le dépôt, il devient visible dans le code et reste exposé dans l'historique Git (même si on le supprime dans un commit ultérieur, il faut alors le révoquer immédiatement). On choisit un Secret pour toute donnée sensible (token API, mot de passe, clé privée) qui doit être chiffrée et masquée, et une Variable (vars) pour les paramètres de configuration non sensibles (nom d'environnement comme staging, URL publique, options de build) qu'il est utile de lire en clair dans les logs.

Question 9 (Partie 6 — Pipeline A) : Quel était le bug, comment l'avez-vous trouvé, comment l'éviter à l'avenir ?

• Bug : Erreur d'indentation YAML à la ligne 12 (un espace en trop devant « - run: pip install -r requirements-dev.txt »), ce qui empêche la lecture syntaxique du fichier de workflow.
• Détection : Le workflow échoue dès le déclenchement dans GitHub Actions avec une erreur de syntaxe YAML indiquant la ligne 12.
• Prévention : Utiliser une extension YAML ou un linter (comme actionlint) dans son éditeur de code pour repérer les erreurs d'indentation avant de commiter.

Question 10 (Partie 6 — Pipeline B) : Quel était le bug, comment l'avez-vous trouvé, comment l'éviter à l'avenir ?

• Bug : Absence de l'étape « - uses: actions/checkout@v6 » au début du job, donc le runner démarre dans un dossier vide sans le code ni le fichier requirements-dev.txt.
• Détection : L'étape pip install échoue dans les logs avec l'erreur « ERROR: Could not open requirements file: [Errno 2] No such file or directory: 'requirements-dev.txt' ».
• Prévention : Toujours commencer les étapes d'un job qui manipule les fichiers du dépôt par actions/checkout@v6 (ou partir d'un modèle de workflow validé).

Question 11 (Partie 6 — Pipeline C) : Quel était le bug, comment l'avez-vous trouvé, comment l'éviter à l'avenir ?

• Bug : Les versions dans la matrix « [3.10, 3.11, 3.12] » ne sont pas entourées de guillemets ; en YAML, le nombre flottant 3.10 est évalué en 3.1, et actions/setup-python cherche alors Python 3.1 (introuvable) au lieu de Python 3.10.
• Détection : Dans la liste des jobs et les logs de setup-python, le job s'appelle « test (3.1) » et échoue car la version 3.1 de Python n'est pas disponible sur le runner.
• Prévention : Toujours écrire les numéros de version entre guillemets dans les fichiers YAML (ex. ["3.10", "3.11", "3.12"]) pour forcer leur interprétation comme chaînes de caractères.

4. Difficultés rencontrées

La mise en place de la matrice de versions Python et du cache pip a nécessité de porter attention au typage YAML des versions de Python (utilisation obligatoire de guillemets autour de "3.11", "3.12" et "3.13") ainsi qu'à l'exécution d'un second run pour observer concrètement la restauration du cache.

Dossier de rendu — GitOps & CI/CD

Auteur : Maxime MAILLARY • **Date :** Novembre 2026 • **Dépôt GitHub :** github.com/maximemlly/tasks-api

---

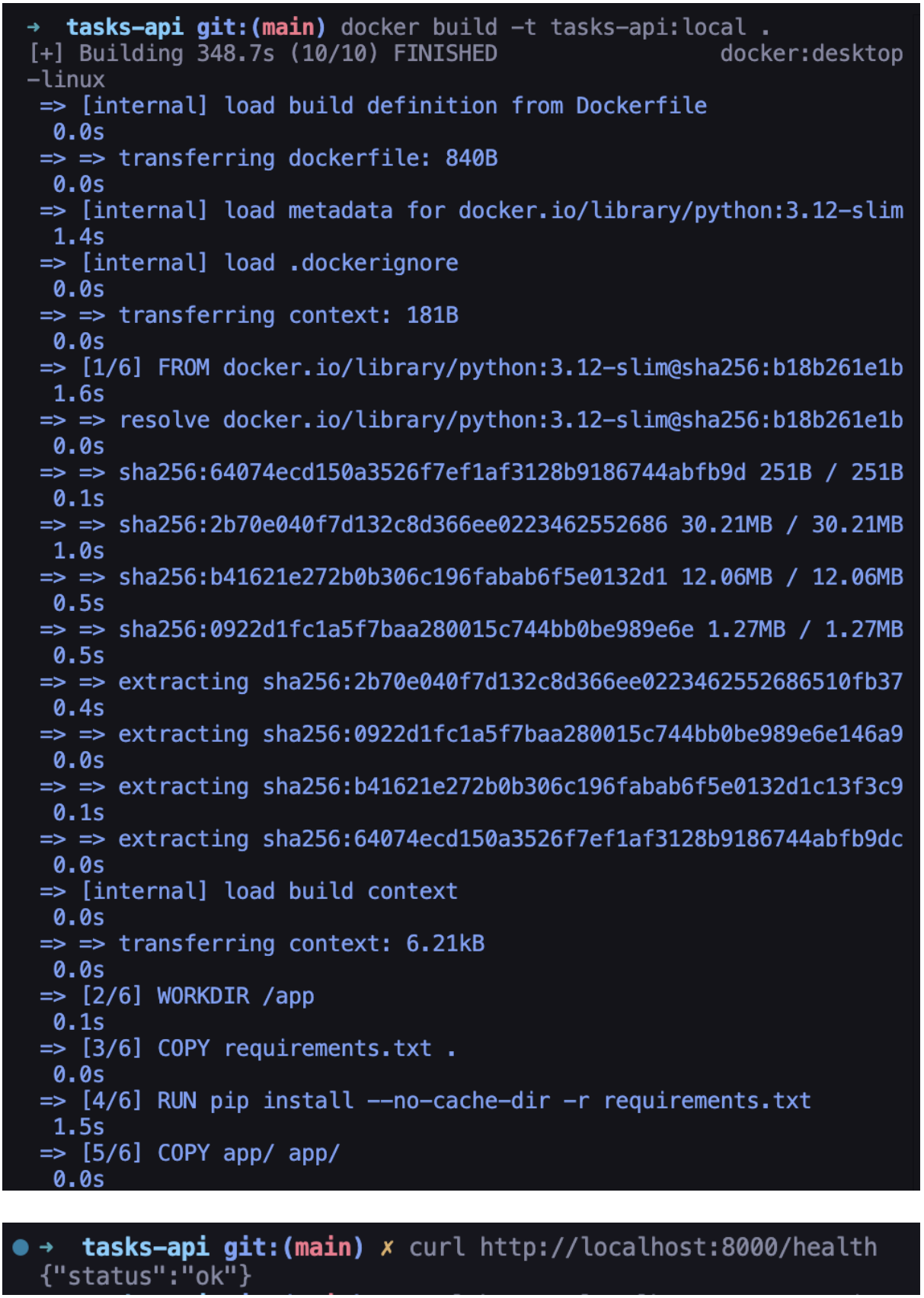
TP 3 : Docker et livraison continue

1. Objectif du TP

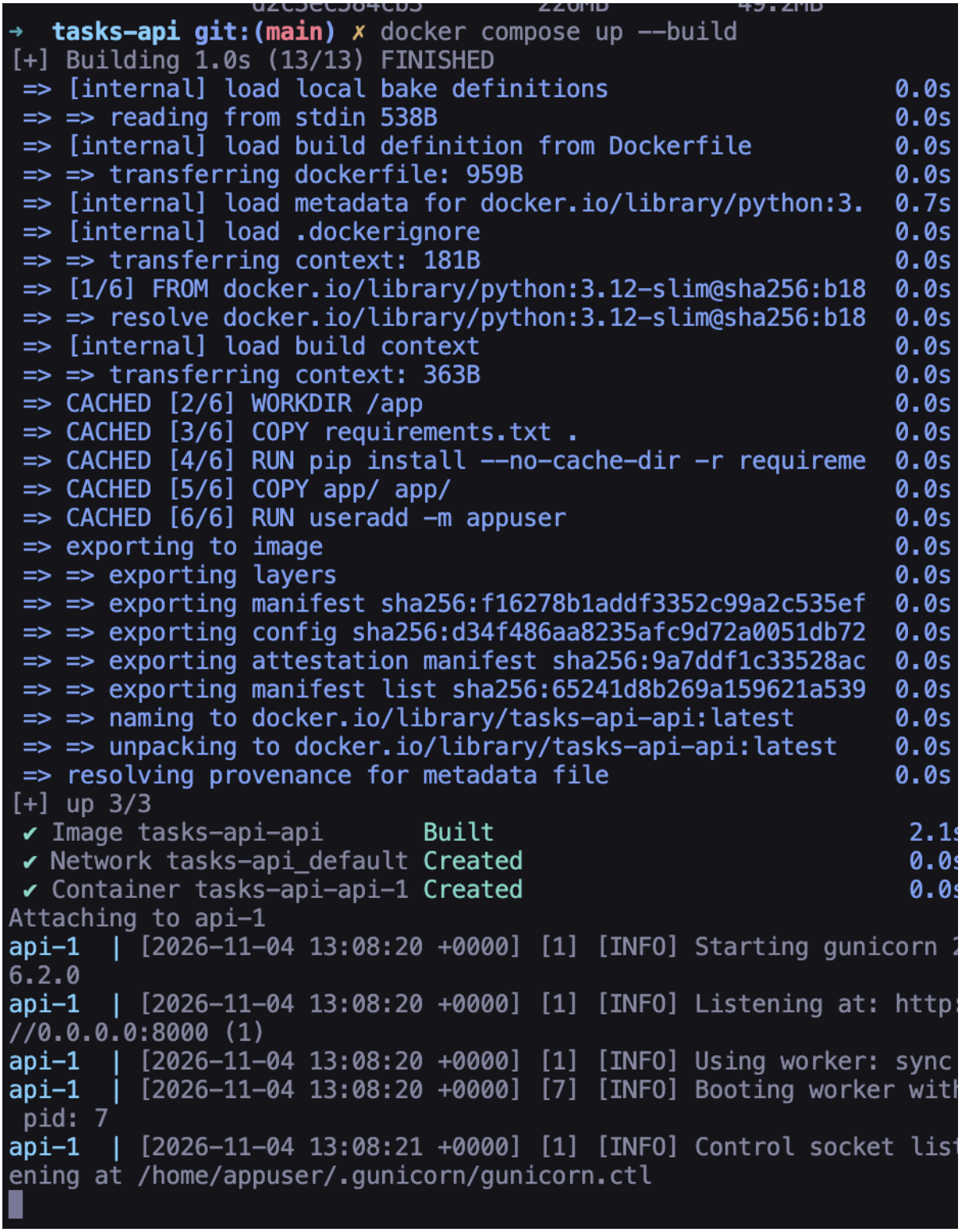
Ce troisième TP consiste à conteneuriser l'application tasks-api à l'aide d'un Dockerfile optimisé et sécurisé (utilisateur non-root, cache des couches) ainsi que d'un fichier Docker Compose pour le développement local. Il met ensuite en place un pipeline de livraison continue (CD) dans GitHub Actions qui publie automatiquement les images versionnées sur GitHub Container Registry (GHCR), crée des releases par tags Git (v0.1.0, v0.1.1) et exécute des tests de fumée sur les environnements staging et production avec approbation manuelle obligatoire.

2. Captures d'écran

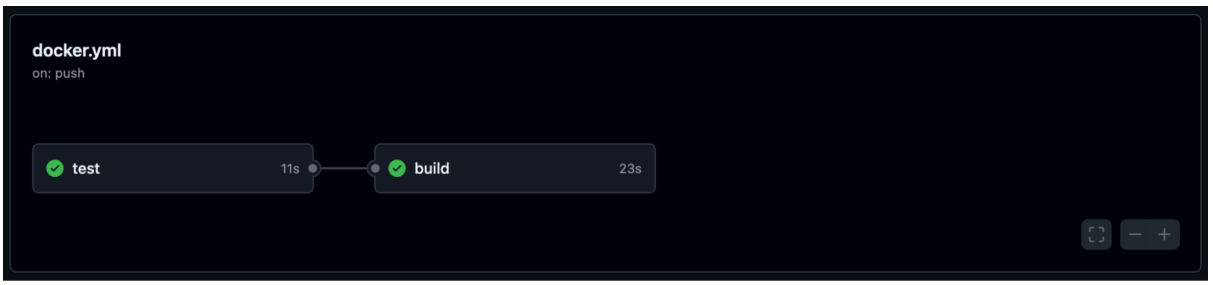
Capture TP3-1 : Construction réussie de l'image locale tasks-api:local avec docker build, puis appel curl sur /health répondant {"status":"ok"}.



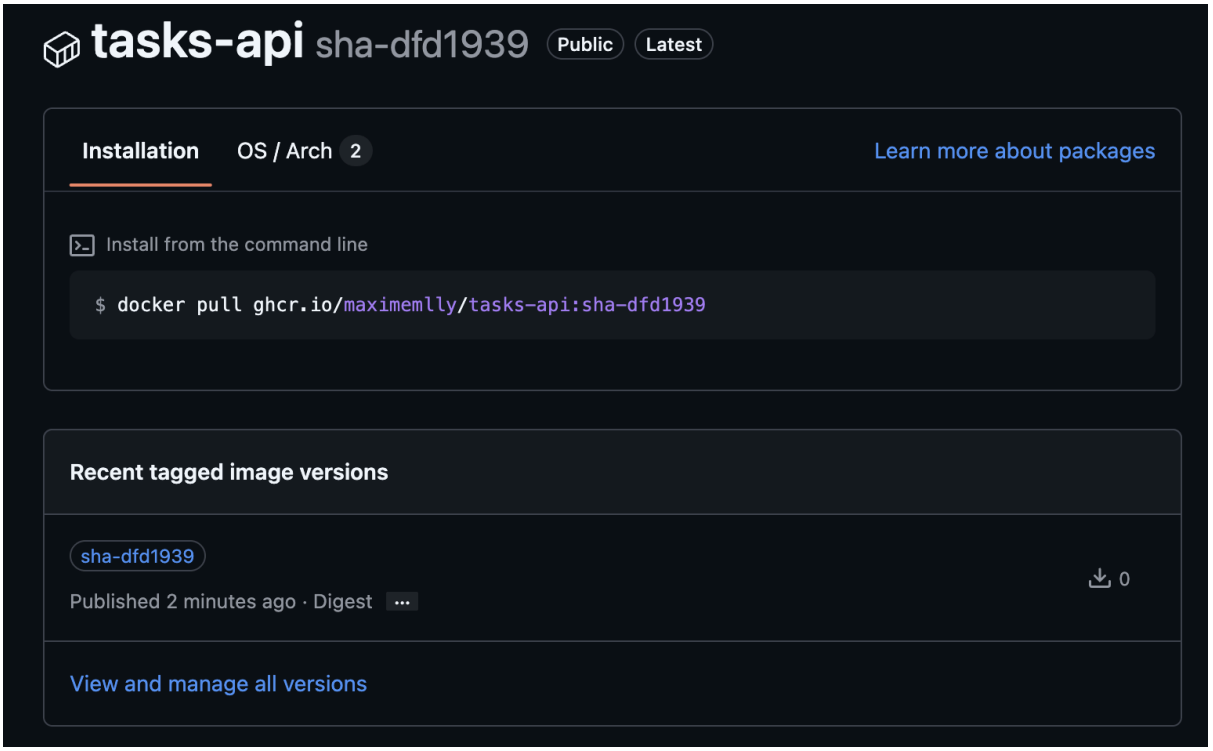
Capture TP3-2 : Lancement du service api avec docker compose up --build (étapes 2 à 6 en cache) et vérification de /version répondant {"version":"local"}.



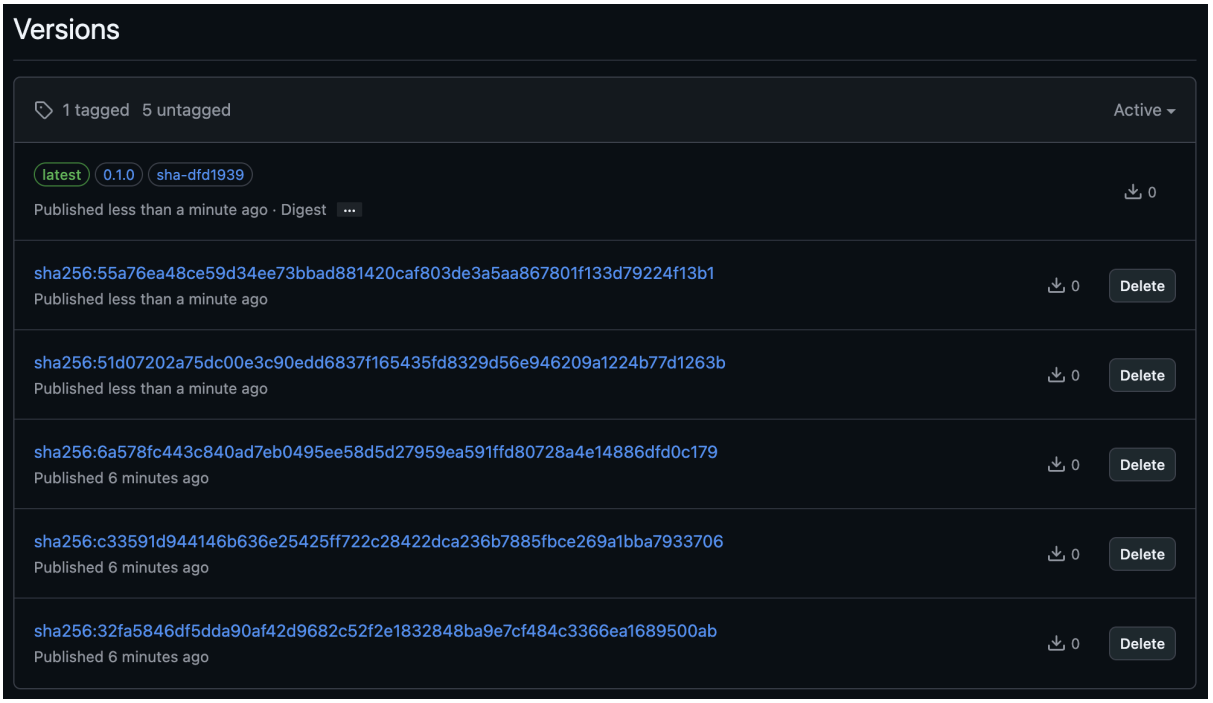
Capture TP3-3 : Exécution réussie du workflow docker.yml dans GitHub Actions avec le job test (11 s) suivi du job build (23 s) au vert.



Capture TP3-4 : Page du package GHCR tasks-api affichant l'image publiée avec son tag de commit sha-dfd1939.

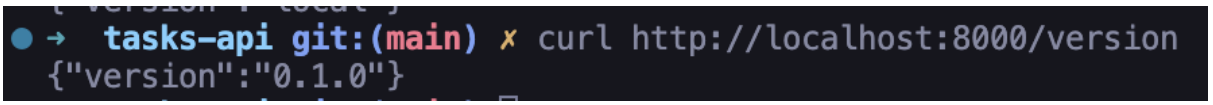


Capture TP3-5 : Liste des versions du package tasks-api sur GHCR montrant la release 0.1.0 ainsi que le tag sha-dfd1939.

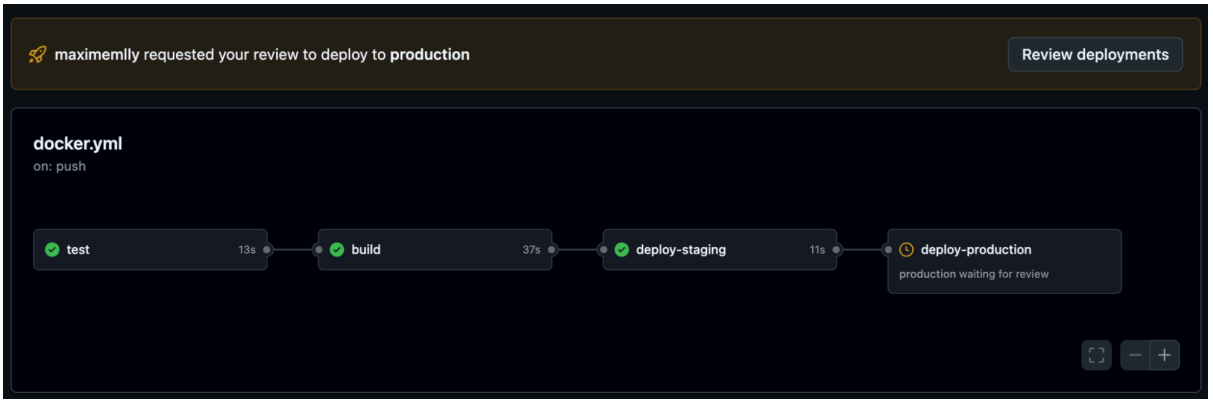


Capture TP3-6 : Téléchargement et exécution avec docker run de l'image ghcr.io/maximemlly/tasks-api:0.1.0 depuis GHCR, et appel curl sur /version répondant {"version":"0.1.0"}.

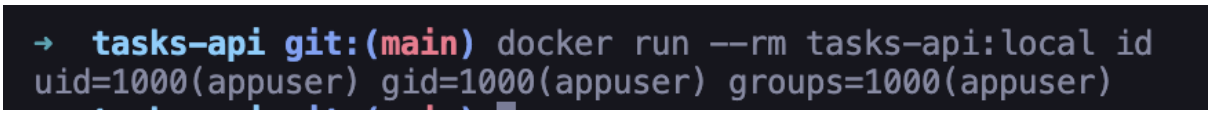
```
→  tasks-api git:(main) docker run --rm --platform linux/amd64 -p 8
000:8000 ghcr.io/maximemlly/tasks-api:0.1.0
Unable to find image 'ghcr.io/maximemlly/tasks-api:0.1.0' locally
0.1.0: Pulling from maximemlly/tasks-api
e541a0dab15d: Pull complete
3eb771151506: Pull complete
acca7542dfc7: Pull complete
ead810bef6cf: Pull complete
6fc242d221a6: Pull complete
5541588704f2: Pull complete
50ec9a50e16d: Pull complete
f574edac59d9: Pull complete
c71dcff06fd3: Pull complete
44136fa355b3: Already exists
b177a0a286e8: Download complete
Digest: sha256:e1d675610f48138762b841b334ffa5fc0aec7b74bedd90c190b3
11154b195e2b
Status: Downloaded newer image for ghcr.io/maximemlly/tasks-api:0.1
.0
[2026-11-04 13:22:58 +0000] [1] [INFO] Starting gunicorn 26.2.0
[2026-11-04 13:22:58 +0000] [1] [INFO] Listening at: http://0.0.0.0
:8000 (1)
[2026-11-04 13:22:58 +0000] [1] [INFO] Using worker: sync
[2026-11-04 13:22:58 +0000] [8] [INFO] Booting worker with pid: 8
[2026-11-04 13:22:58 +0000] [1] [INFO] Control socket listening at
/home/appuser/.gunicorn/gunicorn.ctl
```



Capture TP3-7 : Run du workflow docker.yml sur le tag v0.1.1 avec deploy-staging validé (11 s) et deploy-production en attente d'approbation manuelle (« waiting for review » / « Review deployments »).



Capture TP3-8 : Sortie de la commande docker run --rm tasks-api:local id confirmant que le conteneur s'exécute sous l'utilisateur non-root appuser (uid=1000).



3. Réponses aux questions

Question 1 (Partie 1) : Quelle est la taille de l'image tasks-api ? Lorsqu'on modifie un fichier de app/ et qu'on reconstruit, quelles étapes sont « CACHED » et pourquoi requirements.txt est-il copié avant le code ?

L'image tasks-api:local pèse environ 226 Mo (taille disque affichée par docker images, pour environ 49,2 Mo de contenu compressé) grâce à l'utilisation de l'image de base légère python:3.12-slim. Lorsqu'on modifie uniquement un fichier dans app/, les étapes 2 (WORKDIR /app), 3 (COPY requirements.txt .) et 4 (RUN pip install --no-cache-dir -r requirements.txt) portent la mention « CACHED ». On copie requirements.txt avant le code applicatif car le code change très souvent alors que les dépendances changent rarement : Docker invalide le cache d'une couche (et de toutes les suivantes) dès qu'un fichier copié est modifié. Copier requirements.txt en premier évite donc de réinstaller toutes les bibliothèques pip à chaque modification du code.

Question 2 (Partie 1) : Quelle est la différence entre une image et un conteneur ? À quoi sert le fichier .dockerignore ?

Une image Docker est un modèle immuable (en lecture seule) contenant le système de fichiers, l'environnement d'exécution (Python, dépendances) et le code de l'application, tandis qu'un conteneur est une instance vivante et isolée lancée à partir de cette image (comme la différence entre une classe et un objet en programmation). Le fichier

.dockerignore sert à exclure du contexte de build les fichiers et dossiers inutiles ou sensibles (.venv, .git, __pycache__, .pytest_cache), ce qui accélère l'envoi du contexte au démon Docker, réduit la taille de l'image et évite d'invalider le cache inutilement.

Question 3 (Partie 2) : Que gagne-t-on à décrire le lancement dans un fichier (docker-compose.yml) plutôt que dans une longue commande ?

Décrire la configuration dans un fichier docker-compose.yml permet de versionner dans Git tous les paramètres d'exécution (construction de l'image, ports exposés, variables d'environnement, réseaux, volumes). Toute l'équipe peut ainsi démarrer et arrêter l'environnement complet de manière reproductible avec une seule commande simple (docker compose up / down), sans risque d'oubli ou d'erreur de frappe dans une longue ligne de commande docker run.

Question 4 (Partie 3) : Pourquoi le job build attend-il test (needs: test) ? Pourquoi éviter le tag latest en production ?

Le job build attend la réussite du job test (via needs: test) pour garantir qu'on ne construit et ne publie jamais sur le registre GHCR une image contenant du code qui ne passe pas les tests unitaires, tout en économisant du temps de calcul et du stockage en cas d'échec. En production, il faut éviter le tag latest car il est mutable (écrasé à chaque nouveau build) : on ne sait pas quelle version exacte du code tourne réellement, les déploiements ne sont pas reproductibles et il devient très difficile d'effectuer un retour arrière (rollback) précis.

Question 5 (Partie 4) : Quel est l'avantage d'une image versionnée par rapport à « le code de ma machine » ?

Une image versionnée (ex. ghcr.io/maximemlly/tasks-api:0.1.0) embarque de façon figée le code exact d'un tag Git, la version de Python et les dépendances système et applicatives testées par la CI. Elle élimine le syndrome du « ça marche sur ma machine » : n'importe quel développeur ou serveur de production exécute strictement le même artefact binaire et peut revenir instantanément à une version antérieure connue (0.1.0) en cas d'incident.

Question 6 (Partie 5) : À quoi sert un test de fumée (smoke test) ? Pourquoi demander une approbation humaine avant la production ?

Un test de fumée (smoke test) consiste à démarrer réellement le conteneur construit et à interroger ses routes essentielles (/health et /version) pour vérifier que l'image démarre sans crasher (serveur Gunicorn opérationnel, port 8000 accessible, variables d'environnement bien prises en compte), ce que les tests unitaires pytest seuls ne vérifient pas. Demander une approbation humaine avant la production permet de garder un contrôle métier et opérationnel (vérifier que le staging est valide, choisir le bon créneau de mise en production et valider le changement par un responsable) avant d'impacter les utilisateurs finaux.

Question 7 (Partie 6) : Pourquoi ne pas exécuter une application en root dans un conteneur ? Quelles bonnes pratiques tirer du cas d'étude de compromission d'une action CI ?

Par défaut, l'utilisateur root (uid=0) dans un conteneur correspond au root du noyau Linux de l'hôte : si un attaquant exploite une vulnérabilité dans l'application ou une fuite de conteneur (container escape), il obtient des privilèges d'administrateur beaucoup plus dangereux que si le processus tourne sous un utilisateur restreint sans privilèges (appuser, uid=1000). Concernant la sécurité de la chaîne CI/CD (cas de compromission d'une action tierce

comme Trivy), les bonnes pratiques essentielles sont : épingler les actions tierces sur un hash de commit précis (SHA complet) plutôt qu'un tag modifiable, appliquer le principe du moindre privilège sur le GITHUB_TOKEN (permissions explicites comme packages: write ou contents: read au niveau du job) et limiter la portée des secrets aux seuls environnements qui en ont besoin.

4. Difficultés rencontrées

Deux difficultés principales ont été rencontrées et résolues durant ce TP :
• Lors de la rédaction initiale du Dockerfile (Partie 1), l'instruction RUN gunicorn ... avait été utilisée à la place de CMD ["gunicorn", ...]. Le serveur démarrait donc pendant l'étape docker build et bloquait la fin de la construction de l'image tasks-api:local. Le remplacement par CMD a permis de différer le lancement de Gunicorn au démarrage du conteneur (docker run).
• Lors du test en local de l'image téléchargée depuis GHCR (Partie 4), Docker a renvoyé l'erreur « no matching manifest for linux/arm64/v8 ». En effet, le runner GitHub Actions (ubuntu-latest) construit une image linux/amd64 alors que le Mac utilise une puce Apple Silicon (arm64). L'ajout de l'option --platform linux/amd64 à la commande docker run a résolu le problème.

Dossier de rendu — GitOps & CI/CD

Auteur : Maxime MAILLARY • **Date :** Novembre 2026 • **Dépôt GitHub :** github.com/maximemlly/tasks-api • **Dépôt Config :** github.com/maximemlly/tasks-config

---

TP 4 : GitOps avec Argo CD

1. Objectif du TP

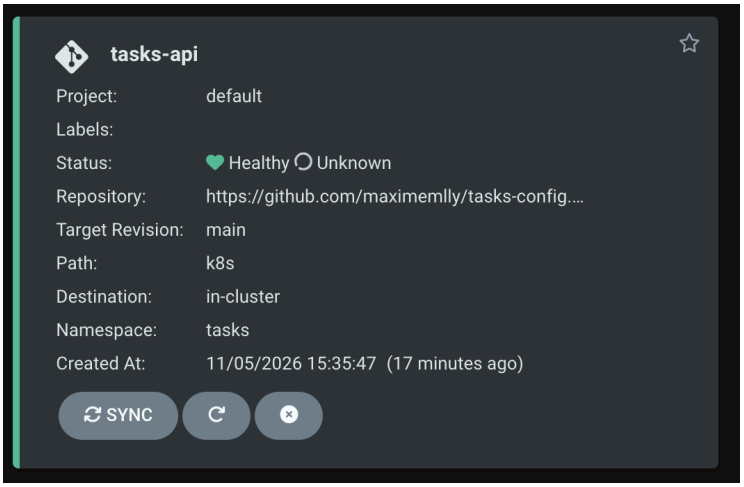
Ce quatrième TP consiste à mettre en œuvre le déploiement continu selon le paradigme GitOps sur un cluster Kubernetes local (kind) piloté par Argo CD. L'état souhaité de l'infrastructure est décrit dans un dépôt Git dédié (tasks-config) séparé du code applicatif (tasks-api) : Argo CD surveille ce dépôt, synchronise automatiquement les ressources Kubernetes, corrige en temps réel toute dérive manuelle (selfHeal), permet des retours arrière immédiats par simple git revert et déploie automatiquement une nouvelle version (v0.2.0) de bout en bout dès qu'un tag Git est poussé.

2. Captures d'écran

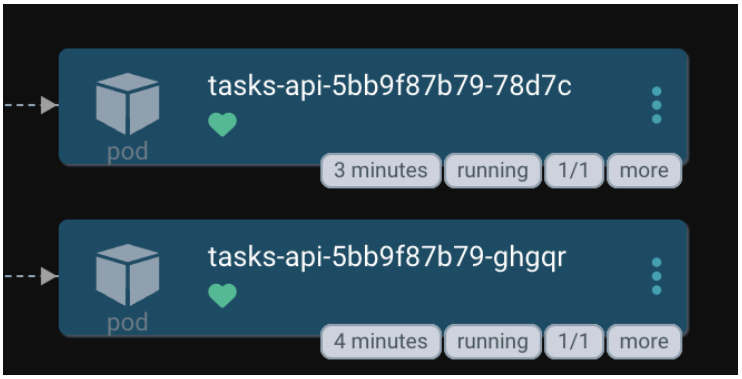
Capture TP4-1 : Vérification du nœud du cluster Kubernetes kind (gitops-control-plane) à l'état Ready avec kubectl get nodes.

```
((.venv) ) → tasks-api git:(main) ✗ kubectl get nodes
NAME                   STATUS   ROLES           AGE   VERSION
gitops-control-plane   Ready    control-plane   75m   v1.37.0
```

Capture TP4-2 : Les 7 pods du namespace argocd à l'état Running et la page de connexion de l'interface Argo CD.



Capture TP4-3 : Application tasks-api déployée dans Argo CD à l'état Synced et Healthy avec ses 2 pods.

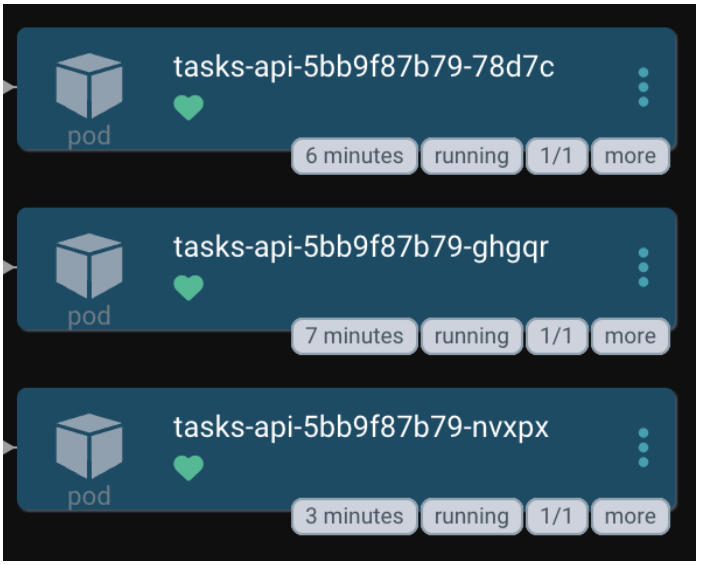


Capture TP4-4 : Appel curl sur http://localhost:8081/version via port-forward répondant {"version":"0.1.0"}.

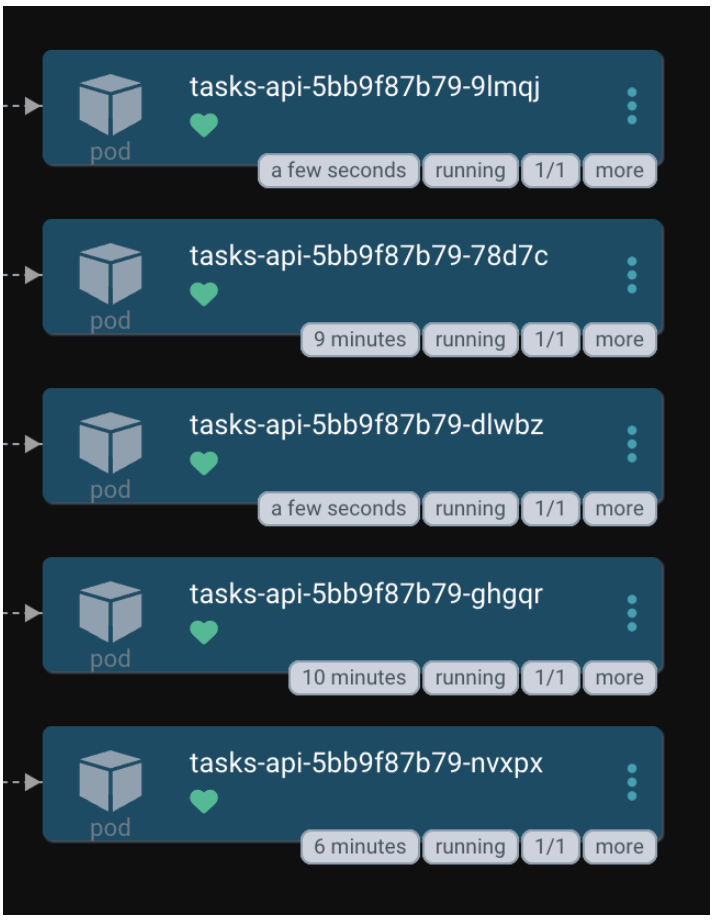
```
[→  tasks-config git:(main) ✗ curl http://localhost:8081/version
{"version":"0.1.0"}
```

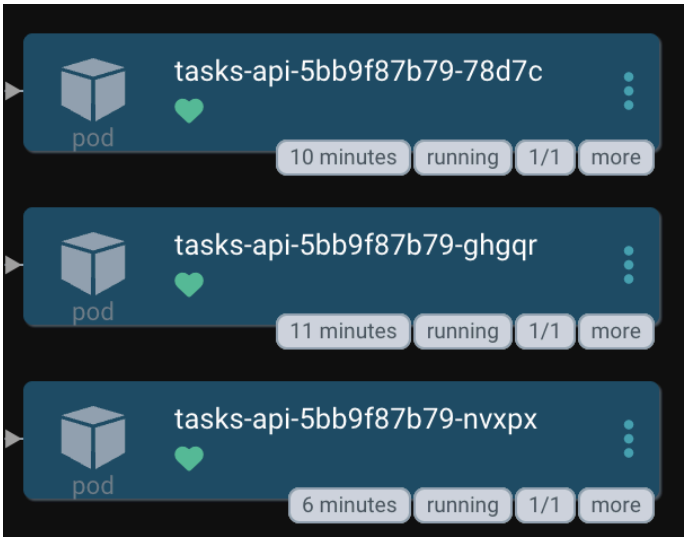
Capture TP4-5 : Commit passant replicas de 2 à 3 dans tasks-config et apparition du 3e pod dans Argo CD.

```
8        -    replicas: 2
     8   +    replicas: 3
```



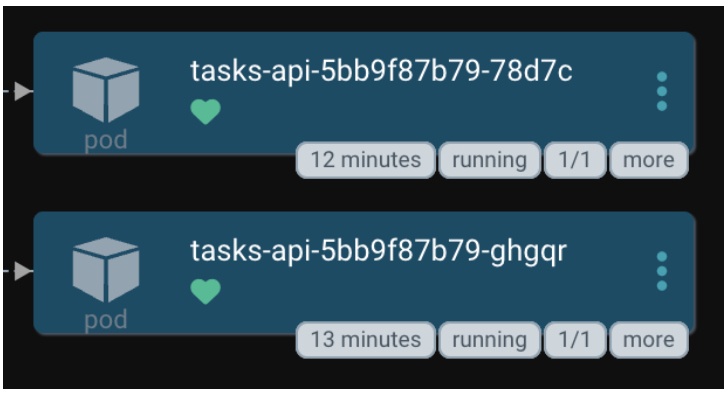
Capture TP4-6 : Passage manuel à 5 pods avec kubectl scale (dérive), puis correction automatique à 3 pods par le selfHeal d'Argo CD.



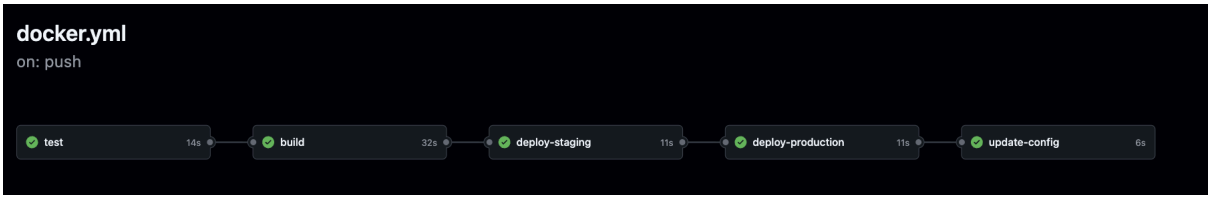


Capture TP4-7 : Historique des commits de tasks-config avec le revert et retour automatique à 2 pods dans Argo CD.



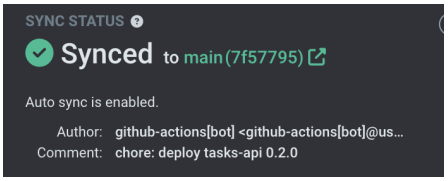


Capture TP4-8 : Exécution complète au vert du workflow docker.yml sur le tag v0.2.0, incluant le job update-config.

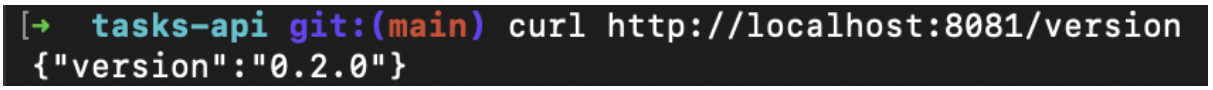


Capture TP4-9 : Commit automatique (chore: deploy tasks-api 0.2.0) dans tasks-config et application Argo CD synchronisée sur 0.2.0.





Capture TP4-10 : Appel curl sur http://localhost:8081/version dans le cluster répondant {"version":"0.2.0"}.



3. Réponses aux questions

Question 1 (Partie 1) : C'est quoi un pod ? Un deployment ? Pourquoi un namespace ?
Un Pod est la plus petite unité déployable dans Kubernetes : il encapsule un ou plusieurs conteneurs (ici notre conteneur tasks-api) qui partagent la même adresse IP, le même espace réseau et le même stockage. Un Deployment est un contrôleur déclaratif de plus haut niveau qui gère le cycle de vie d'un ensemble de pods : il maintient en permanence le nombre d'instances souhaité (replicas: 2), recrée automatiquement un pod en cas de crash et orchestre les mises à jour progressives sans interruption de service (rolling updates). Enfin, un namespace permet de cloisonner et d'isoler logiquement les ressources au sein d'un même cluster physique (ici séparer les composants système d'administration dans argocd et les ressources de notre application métier dans tasks).

Question 2 (Partie 2) : Pourquoi sépare-t-on le dépôt de l'application et le dépôt de configuration ?
Séparer le dépôt applicatif (tasks-api) et le dépôt de configuration Kubernetes (tasks-config) apporte trois avantages majeurs : premièrement, cela découple les cycles de vie CI et CD (modifier le nombre de réplicas ou une ressource Kubernetes dans tasks-config ne relance pas inutilement les tests Python ni le build de l'image Docker dans tasks-api) ; deuxièmement, cela permet d'isoler les droits d'accès (les développeurs poussent sur le dépôt applicatif tandis que seuls le pipeline de release et l'équipe Ops modifient le dépôt d'infrastructure) ; troisièmement, l'historique Git de tasks-config reste un journal d'audit pur où chaque commit correspond exactement à un changement d'état du cluster.
Question 3 (Partie 3) : Que signifie selfHeal ? Pourquoi le retour arrière est-il simple en GitOps ?
L'option selfHeal: true (auto-réparation) signifie qu'Argo CD compare en continu l'état réel du cluster Kubernetes avec l'état cible décrit dans Git : si une modification manuelle est effectuée directement sur le cluster (comme kubectl scale --replicas=5), Argo CD détecte immédiatement cette dérive (configuration drift) et écrase le changement manuel pour rétablir l'état déclaré dans Git (replicas: 3). Le retour arrière (rollback) est très simple en GitOps car Git constitue l'unique source de vérité : il suffit d'annuler le commit dans le dépôt de configuration avec git revert HEAD puis git push pour qu'Argo CD applique automatiquement l'état antérieur, sans jamais manipuler le cluster à la main.
Question 4 (Partie 4) : Résumez en 5 lignes le chemin d'un git tag jusqu'au pod. Qui pousse (push) et qui tire (pull) ? Quelle différence avec un déploiement CI/CD « classique » ?
1. Le développeur crée et pousse le tag v0.2.0 sur le dépôt applicatif tasks-api, ce qui déclenche le workflow docker.yml.
2. Les jobs test et build exécutent pytest puis construisent et poussent l'image ghcr.io/maximemlly/tasks-api:0.2.0 sur le registre GHCR.
3. Le job deploy-staging valide l'image par un test de fumée, puis deploy-production attend l'approbation manuelle du reviewer avant de valider la release.
4. Une fois la production approuvée, le job update-config clone tasks-config à l'aide du secret CONFIG_REPO_TOKEN, remplace le tag par 0.2.0 dans k8s/deployment.yaml via scripts/set-image-tag.sh, puis commite et pousse sur main.
5. Argo CD détecte ce nouveau commit dans tasks-config, synchronise le manifeste Deployment dans le namespace tasks, et Kubernetes remplace les pods par la version 0.2.0.
Dans ce flux, le développeur et le pipeline GitHub Actions poussent (push) le code, l'image Docker vers GHCR et le commit de configuration vers Git, tandis qu'Argo CD et le cluster Kubernetes tirent (pull) les manifestes depuis Git et l'image depuis GHCR. Contrairement à un déploiement CI/CD « classique » (push-based) où le runner CI exécute kubectl apply depuis l'extérieur et doit donc détenir les clés d'administration du cluster, en GitOps (pull-based) l'agent Argo CD tourne à l'intérieur du cluster : le cluster n'expose pas son API d'administration à la CI et se réconcilie de manière autonome.

4. Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées et résolues au cours de ce TP :
• Installation et démarrage de kind sur macOS (Partie 1) : La commande kind create cluster --name gitops échouait initialement car l'outil kind n'était pas encore installé (installation via

brew install kind) et Docker Desktop devait être lancé au préalable pour fournir le démon Docker nécessaire aux nœuds du cluster.
• Incompatibilité d'architecture d'image amd64 / arm64 dans kind (Partie 2) : Lors du premier déploiement par Argo CD, les pods restaient bloqués en ImagePullBackOff avec l'erreur « no match for platform in manifest: not found », car le runner GitHub Actions (ubuntu-latest) avait construit l'image en linux/amd64 alors que le nœud kind sur Mac Apple Silicon tourne en linux/arm64. Le chargement de l'image compilée pour arm64 dans le cluster (kind load docker-image) a permis aux pods de passer immédiatement à l'état Running.
• Réactivité quasi instantanée du selfHeal (Partie 3) : Lors du passage manuel à 5 réplicas (kubectl scale --replicas=5), Argo CD détectait et corrigeait la dérive en moins d'une seconde, ramenant déjà les pods à 3 avant l'affichage de kubectl get pods. Il a fallu observer les événements du namespace et suspendre temporairement selfHeal pour capturer l'état intermédiaire à 5 pods.
• Configuration du secret CONFIG_REPO_TOKEN (Partie 4) : Le premier lancement du job update-config a échoué sur l'étape actions/checkout avec l'erreur « Input required and not supplied: token ». La création d'un Fine-grained Personal Access Token limité au dépôt tasks-config (permission Contents: Read and write) et son ajout dans les Repository secrets de tasks-api sous le nom CONFIG_REPO_TOKEN ont permis au pipeline de pousser automatiquement le commit de la version 0.2.0.

5. Bilan de la semaine

Ces quatre TP m'ont permis de construire progressivement une chaîne CI/CD et GitOps complète, depuis l'exécution automatisée de pytest à chaque push (TP1) jusqu'au déploiement continu dans Kubernetes piloté par Argo CD (TP4). J'ai retenu l'importance de structurer le pipeline en étapes complémentaires : vérification du style (flake8, black), tests multi-versions en matrice avec seuil de couverture et cache pip (TP2), puis conteneurisation sécurisée (utilisateur non-root) avec tests de fumée et approbation manuelle avant la production (TP3). La mise en pratique du modèle GitOps pull-based en fin de parcours a illustré concrètement l'intérêt de séparer le dépôt applicatif (tasks-api) du dépôt d'état déclaratif (tasks-config), tant pour la sécurité que pour l'auto-réparation (selfHeal) et la simplicité des retours arrière par git revert. Les principales difficultés ont porté sur l'indentation stricte des workflows YAML, la gestion des permissions inter-dépôts (GITHUB_TOKEN, CONFIG_REPO_TOKEN) et les différences d'architecture processeur (amd64 sur les runners GitHub Actions contre arm64 sur Mac Apple Silicon). Si je devais refaire ce projet, je configurerais dès le TP3 une construction d'image Docker multi-architecture (linux/amd64,linux/arm64) avec Buildx afin d'éviter tout problème de compatibilité sur le cluster local.